

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 1

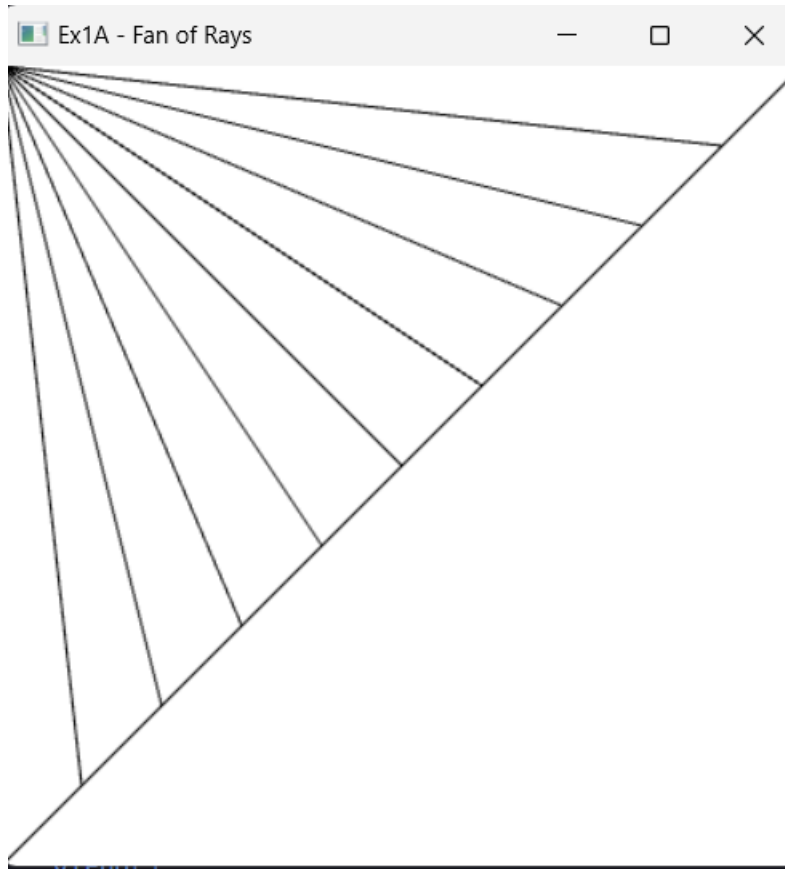
Part 1:

Ex 1A – Fan of Rays:

Code:

```
1  # Exercise 1A Starter - Fan of Rays from top-left to diagonal points
2  from window_glut import Simple2DApp
3  from OpenGL.GLUT import glutMainLoop
4  from OpenGL.GL import *
5  from OpenGL.GLU import *
6  import numpy as np
7
8  class App(Simple2DApp):
9      def draw(self):
10         W, H = self.width, self.height
11         N = 9 # TODO: make configurable
12
13         # TODO: Loop i in [1..N], compute t = i/(N+1), and draw one ray per i
14         # self.draw_line(0, H, W*t, H*t, color=(0,0,0), width=1)
15         for i in range(1, N+1):
16             t = i/(N+1)
17             self.draw_line(0, H, W*t, H*t, color=(0,0,0), width=1)
18
19         # TODO: draw the diagonal
20         # self.draw_line(0, 0, W, H, color=(0,0,0), width=1)
21         self.draw_line(0, 0, W, H, color=(0,0,0), width=1)
22
23 if __name__ == "__main__":
24     App(400, 400, b"Ex1A - Fan of Rays")
25     glutMainLoop()
26
```

Result:

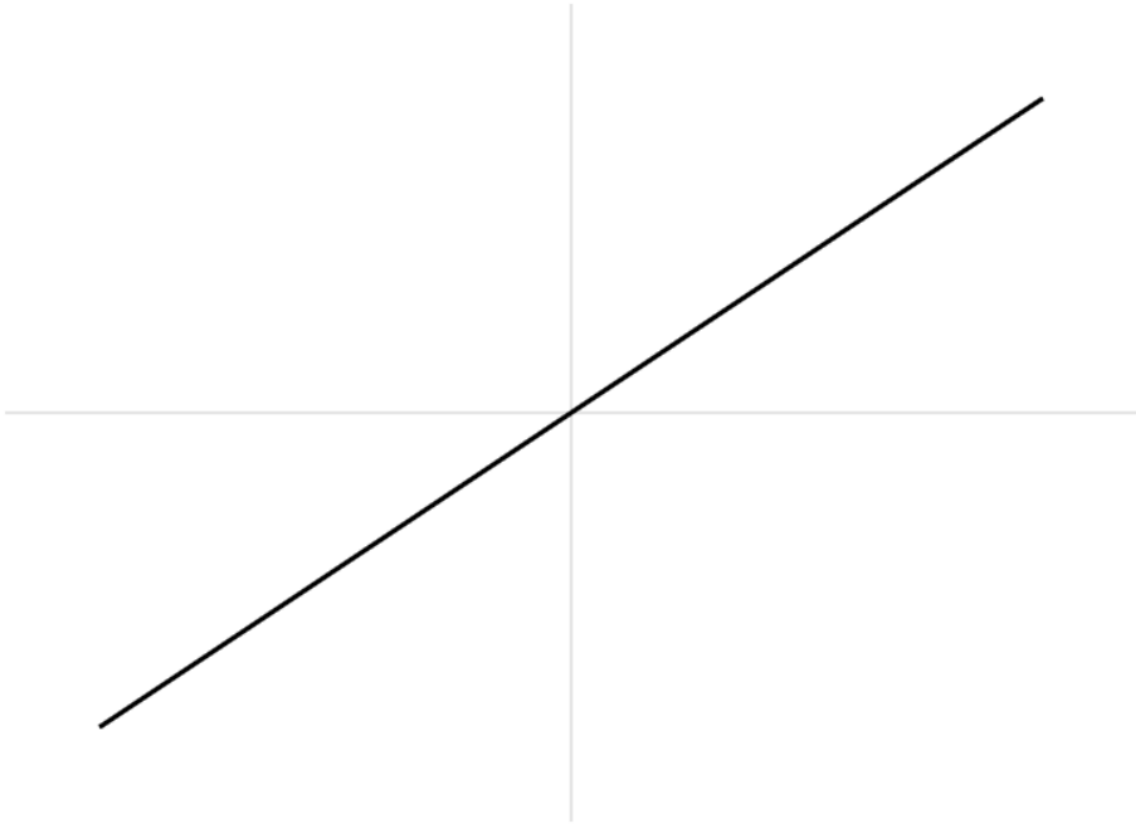


Ex 1B – Axes + Main Line

Code:

```
1  # Exercise 1B Starter - Axes + Main Line
2  from window_glut import Simple2DApp
3  from OpenGL.GLUT import glutMainLoop
4
5  class App(Simple2DApp):
6      def draw(self):
7          W, H = self.width, self.height
8          cx, cy = W//2, H//2
9
10         # TODO: axes
11         # self.draw_line(40, cy, W-40, cy, color=(0.8,0.8,0.8), width=1)
12         self.draw_line(40, cy, W-40, cy, color=(0.8,0.8,0.8), width=1)
13         # self.draw_line(cx, 40, cx, H-40, color=(0.8,0.8,0.8), width=1)
14         self.draw_line(cx, 40, cx, H-40, color=(0.8,0.8,0.8), width=1)
15         # TODO: main line (thicker)
16         # self.draw_line(100, 100, W-100, H-120, color=(0,0,0), width=3)
17         self.draw_line(100, 100, W-100, H-120, color=(0,0,0), width=3)
18
19 if __name__ == "__main__":
20     App(800, 600, b"Ex1B - Axes + Line")
21     glutMainLoop()
22
```

Result:

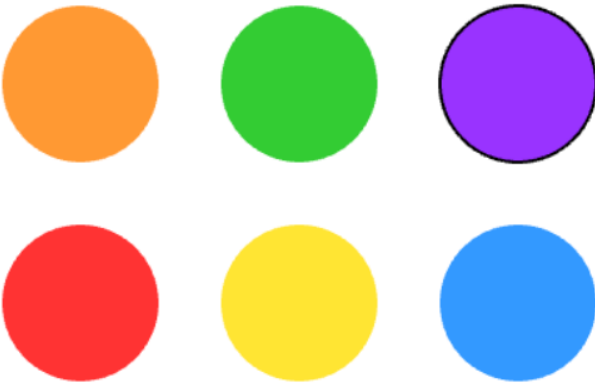
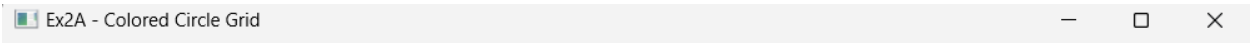


Ex2_1_circles_fill.py

Code:

```
1  # Exercise 2A Starter - Colored Filled Circles (3x2 grid)
2  from window_glut import Simple2DApp
3  from OpenGL.GLUT import glutMainLoop
4
5  class App(Simple2DApp):
6      def draw(self):
7          # Suggested grid parameters
8          W, H = self.width, self.height
9          dx, dy = 140, 140
10         r = 50
11         cx0, cy0 = W//2 - dx, 120
12         colors = [
13             (1.0, 0.2, 0.2), (1.0, 0.6, 0.2), (1.0, 0.9, 0.2),
14             (0.2, 0.8, 0.2), (0.2, 0.6, 1.0), (0.6, 0.2, 1.0),
15         ]
16
17         # TODO: nested loop, use k to pick colors[k % len(colors)]
18         # x = cx0 + col*dx - r; y = cy0 + row*dy - r
19         for col in range(3):
20             for row in range(2):
21                 k = col * 2 + row
22                 k = colors[k % len(colors)]
23                 x = cx0 + col*dx - r; y = cy0 + row*dy - r
24                 self.draw_oval_filled(x, y, 2*r, 2*r, color=k)
25                 # self.draw_oval_filled(x, y, 2*r, 2*r, color=...)
26                 # self.draw_oval_outline(x, y, 2*r, 2*r, color=(0,0,0), width=2)
27                 self.draw_oval_outline(x, y, 2*r, 2*r, color=(0,0,0), width=2)
28
29 if __name__ == "__main__":
30     App(800, 600, b"Ex2A - Colored Circle Grid")
31     glutMainLoop()
32
```

Result:

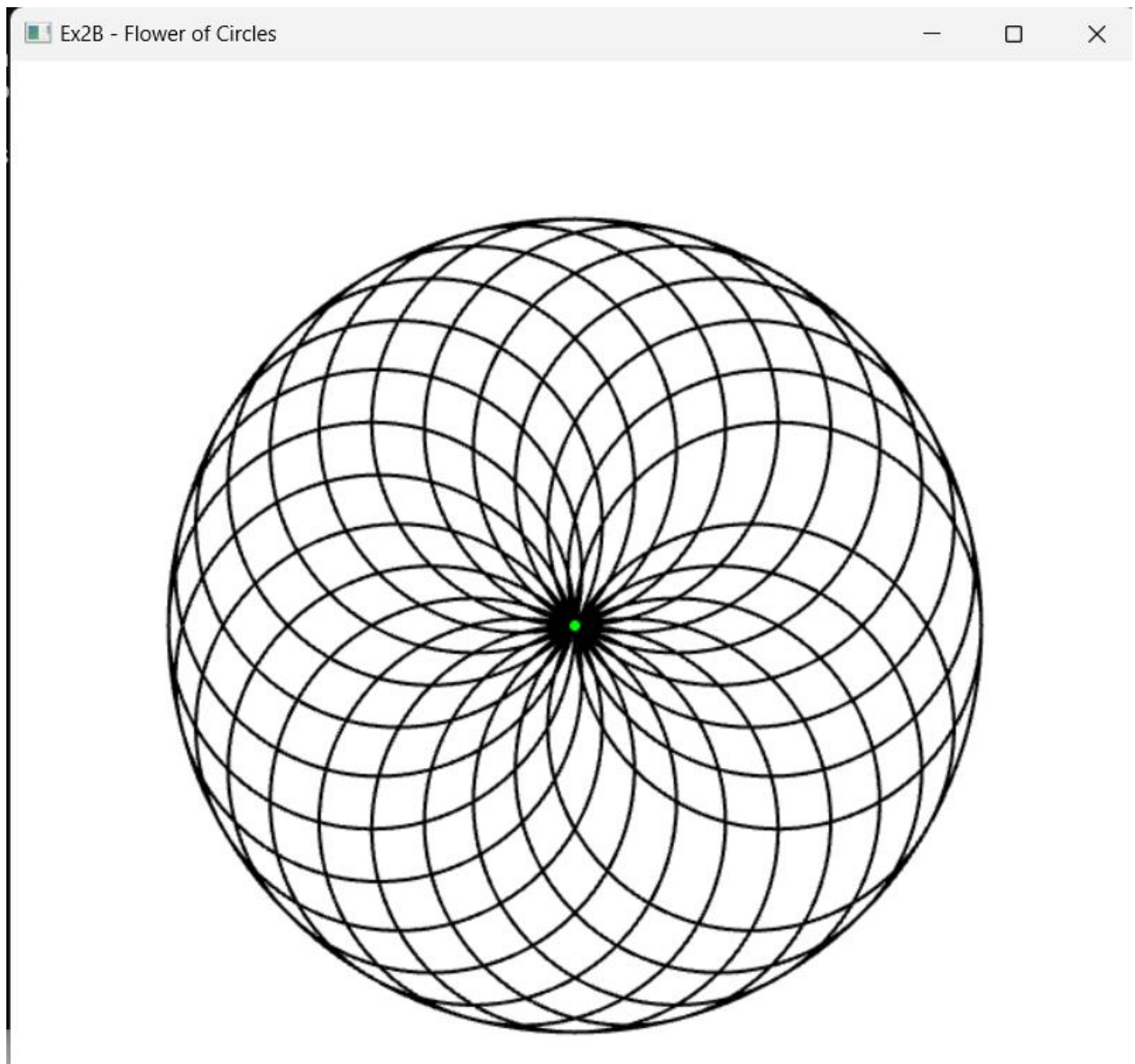


Ex2_2_set_circles.py

Code:

```
1  # Exercise 2B Starter - Flower of Circles
2  import math
3  from window_glut import Simple2DApp
4  from OpenGL.GLUT import glutMainLoop
5
6  class App(Simple2DApp):
7      def draw(self):
8          W, H = self.width, self.height
9          cx, cy = W*0.5, H*0.5
10         N = 24
11         r = min(W, H)*0.18
12         R = r
13
14         # TODO: Loop k in [0..N-1]: angle th = 2*pi*k/N, center (x,y) = (cx+R*cos(th), cy+R*sin(th))
15         for k in range(0, N-1):
16             th = 2*math.pi*k/N
17             x, y = cx+R*math.cos(th), cy+R*math.sin(th)
18             # self.draw_oval_outline(x - r, y - r, 2*r, 2*r, color=(0,0,0), width=2)
19             self.draw_oval_outline(x - r, y - r, 2*r, 2*r, color=(0,0,0), width=2)
20             # Optional outer boundary and center dot
21             # self.draw_oval_outline(cx - 2*r, cy - 2*r, 4*r, 4*r, color=(0,0,0), width=2)
22             self.draw_oval_outline(cx - 2*r, cy - 2*r, 4*r, 4*r, color=(0,0,0), width=2)
23             # self.draw_oval_filled(cx - 3, cy - 3, 6, 6, color=(0,1,0))
24             self.draw_oval_filled(cx - 3, cy - 3, 6, 6, color=(0,1,0))
25
26 if __name__ == "__main__":
27     App(640, 640, b"Ex2B - Flower of Circles")
28     glutMainLoop()
29
```

Result:

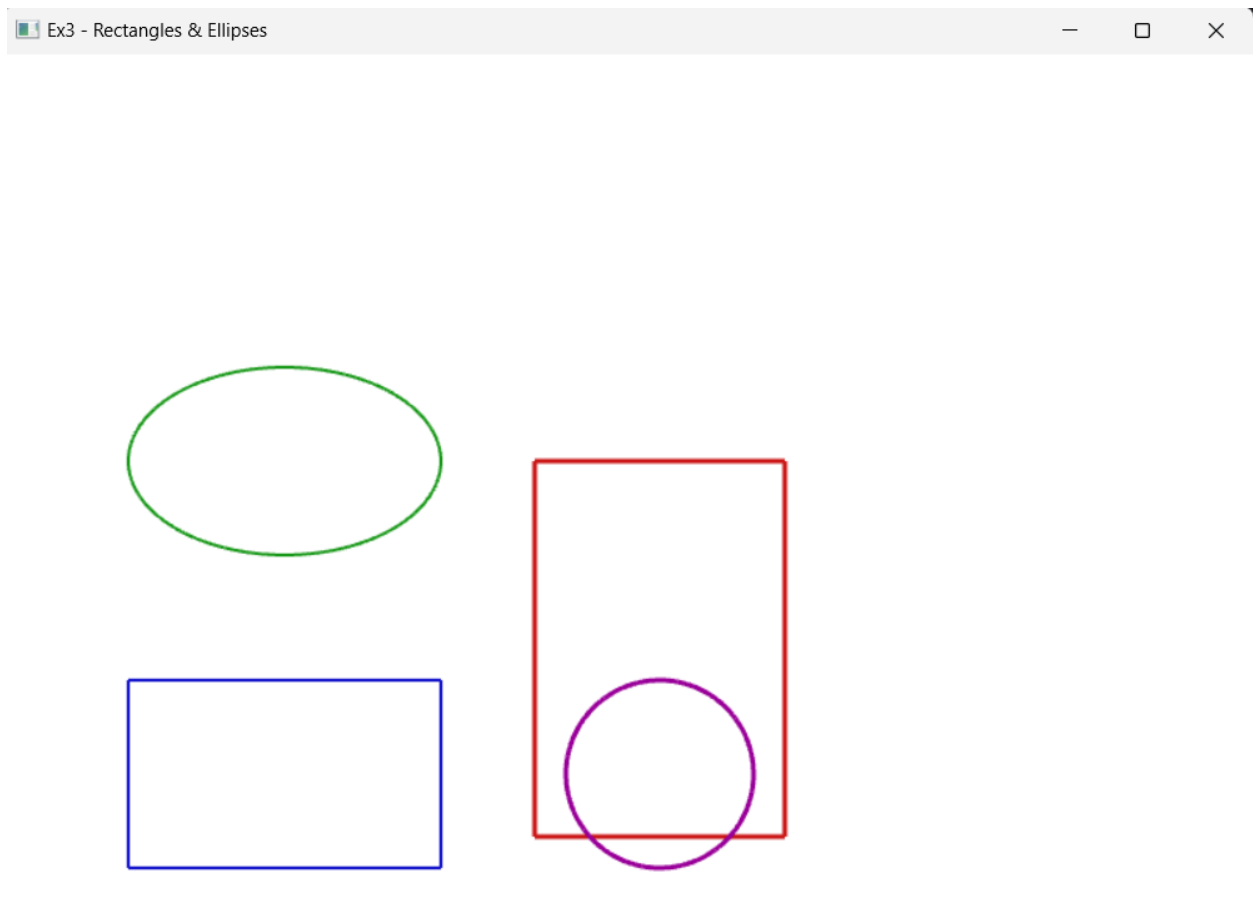


Ex3_1_rect_ellipse.py

Code:

```
1  # Exercise 3 Starter - Rectangles & Ellipses (Outlines)
2  from window_glut import Simple2DApp
3  from OpenGL.GLUT import glutMainLoop
4
5  class App(Simple2DApp):
6      def draw(self):
7          # TODO: at least 2 rectangles and 2 ellipses using outline helpers
8          # self.draw_rect_outline(80, 80, 200, 120, color=(0.1,0.1,0.8), width=2)
9          self.draw_rect_outline(80, 80, 200, 120, color=(0.1,0.1,0.8), width=2)
10         # self.draw_rect_outline(340, 100, 160, 240, color=(0.8,0.1,0.1), width=3)
11         self.draw_rect_outline(340, 100, 160, 240, color=(0.8,0.1,0.1), width=3)
12         # self.draw_oval_outline(80, 280, 200, 120, color=(0.1,0.6,0.1), width=2)
13         self.draw_oval_outline(80, 280, 200, 120, color=(0.1,0.6,0.1), width=2)
14         # self.draw_oval_outline(360, 80, 120, 120, color=(0.6,0,0.6), width=3)
15         self.draw_oval_outline(360, 80, 120, 120, color=(0.6,0,0.6), width=3)
16         pass
17
18 if __name__ == "__main__":
19     App(800, 600, b"Ex3 - Rectangles & Ellipses")
20     glutMainLoop()
21
```

Result:



Ex4_star_and_text.py

Code:

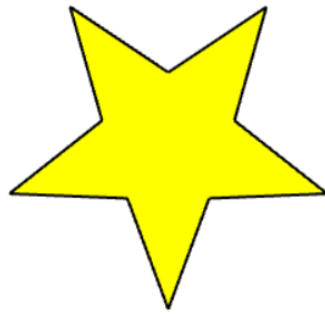


```
1  # Exercise 4A Starter - Star + Caption
2  import math
3  from OpenGL.GL import *
4  from OpenGL.GLUT import glutMainLoop
5  from window_glut import Simple2DApp
6
7  def star_points(cx, cy, R_outer=120, R_inner=50, start_angle_deg=-90):
8      # TODO: compute 5 outer and 5 inner points (alternating ring)
9      points = []
10     for i in range(10):
11         ang = math.radians(start_angle_deg + i*36.0)
12         R = R_outer if i % 2 == 0 else R_inner
13         points.append((cx + R*math.cos(ang), cy + R*math.sin(ang)))
14     # return list_of_10_vertices
15     return points
16
17 class App(Simple2DApp):
18     def draw(self):
19         W,H = self.width, self.height
20         cx, cy = W*0.35, H*0.6
21
22         # TODO: call star_points and build the alternating ring
23         points = star_points(cx,cy)
24         # TODO: fill star with GL_TRIANGLE_FAN (glBegin/glEnd)
25         glColor3f(1.0, 1.0, 0.0)
26         glBegin(GL_TRIANGLE_FAN)
27         glVertex2f(cx, cy)
28         for (x, y) in points:
29             glVertex2f(x, y)
30         glVertex2f(*points[0])
31         glEnd()
32         glFlush()
33         # TODO: draw outline-only star with self.draw_polyline(..., loop=True)
34         self.draw_polyline(points, color=(0,0,0), width=2, loop=True)
35
36         points = star_points(2*cx,cy)
37         self.draw_polyline(points, color=(0,0,255), width=4, loop=True)
38         # TODO: caption with self.draw_text(..., "This is a star")
39         self.draw_text(cx-50, cy-150, "This is a star")
40         self.draw_text(2*cx-50, cy-150, "This is a star")
41
42 if __name__ == "__main__":
43     App(900, 600, b"Ex4A - Star + Text")
44     glutMainLoop()
45
```

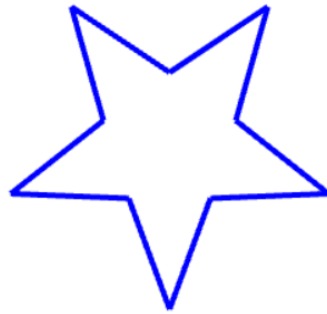
Result:

Ex4A - Star + Text

— □ ×



This is a star



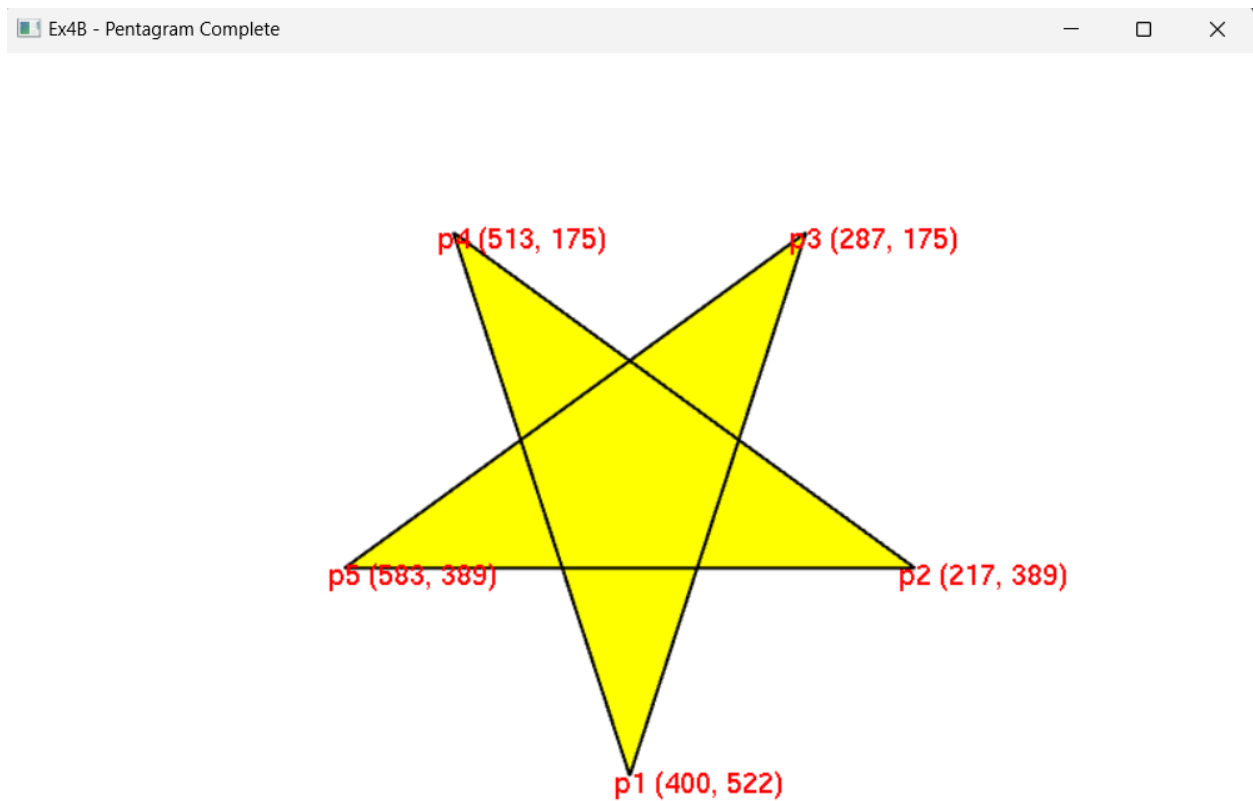
This is a star

Ex4_star_complete.py

Code:

```
1  # Exercise 4B Starter - Pentagram (Complete) + Labels
2  import math
3  from OpenGL.GL import *
4  from OpenGL.GLUT import glutMainLoop
5  from window_glut import Simple2DApp
6
7  def star_points_regular(cx, cy, R):
8      # TODO: compute outer[5] and inner[5] with r = R * sin(18deg)/sin(54deg)
9      r = R * math.sin(math.radians(18)) / math.sin(math.radians(54))
10     outer, inner = [], []
11
12     for i in range(5):
13         angle_outer = math.radians(90 + i*72)
14         angle_inner = math.radians(90 + i*72 + 36)
15         ox = cx + R * math.cos(angle_outer)
16         oy = cy + R * math.sin(angle_outer)
17         ix = cx + r * math.cos(angle_inner)
18         iy = cy + r * math.sin(angle_inner)
19         outer.append((ox, oy))
20         inner.append((ix, iy))
21     return outer, inner
22
23 class App(Simple2DApp):
24     def draw(self):
25         W,H = self.width, self.height
26         cx, cy = W*0.5, H*0.55
27         R = min(W, H) * 0.32
28
29         # TODO: get outer, inner = star_points_regular(cx, cy, R)
30         outer, inner = star_points_regular(cx, cy, R)
31         # TODO: build ring and fill with GL_TRIANGLE_FAN
32         glPushMatrix()
33         glTranslatef(cx, cy, 0)
34         glRotatef(180, 0, 0, 1)
35         glTranslatef(-cx, -cy, 0)
36         glColor3f(1, 1, 0)
37         glBegin(GL_TRIANGLE_FAN)
38         glVertex2f(cx, cy)
39         for i in range(5):
40             glVertex2f(*outer[i])
41             glVertex2f(*inner[i])
42         glVertex2f(*outer[0])
43         glEnd()
44         # TODO: draw pentagram outline using order = [0,2,4,1,3,0]
45         glColor3f(0, 0, 0)
46         glLineWidth(2)
47         order = [0,2,4,1,3,0]
48         glBegin(GL_LINE_STRIP)
49         for i in order:
50             glVertex2f(*outer[i])
51         glEnd()
52         glFlush()
53         # TODO: Label p1..p5 near the outer vertices, and add caption
54         glColor3f(1, 1, 0)
55         for i, (x,y) in enumerate(outer, start=1):
56             label = f"p{i} ({x:.0f}, {y:.0f})"
57             self.draw_text(x+10, y+10, label, color=(1, 0, 0))
58
59 if __name__ == "__main__":
60     App(800, 600, b"Ex4B - Pentagram Complete")
61     glutMainLoop()
62
```

Result:



Ex5_hello2d.py

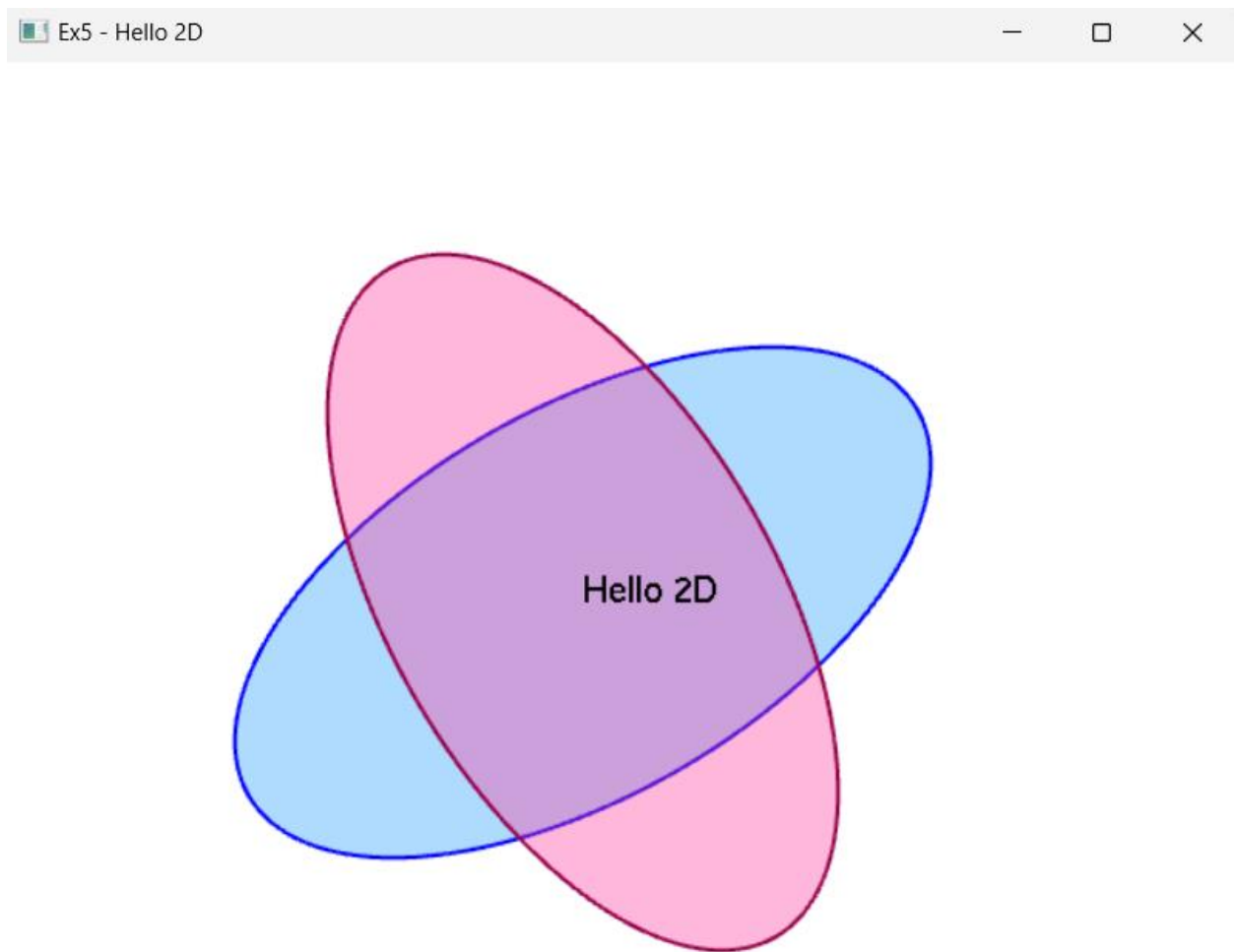
Code:

```

1  # Exercise 5 Starter - Hello 2D Transforms + Filled Ellipses
2  from OpenGL.GL import *
3  from OpenGL.GLUT import glutMainLoop
4  from window_glut import Simple2DApp
5  import math
6
7  class App(Simple2DApp):
8      def draw_ellipse_outline(self, cx, cy, rx, ry, color=(0,0,1), width=2, segments=120):
9          glColor3f(*color)
10         glLineWidth(width)
11         glBegin(GL_LINE_LOOP)
12         for i in range(segments):
13             t = 2.0*math.pi*(i/segments)
14             glVertex2f(cx + rx*math.cos(t), cy + ry*math.sin(t))
15         glEnd()
16
17     def draw_ellipse_filled(self, cx, cy, rx, ry, color=(0,0,1,1), segments=120):
18         # TODO: implement triangle-fan filled ellipse (center first, then rim)
19         glColor4f(*color)
20         glBegin(GL_TRIANGLE_FAN)
21         glVertex2f(cx, cy)
22         for i in range(segments+1):
23             theta = 2.0 * math.pi * i / segments
24             x = cx + rx * math.cos(theta)
25             y = cy + ry * math.sin(theta)
26             glVertex2f(x, y)
27         glEnd()
28         pass
29
30     def draw(self):
31         glEnable(GL_BLEND);
32         glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
33         glPushMatrix()
34         # TODO: apply Translate -> Scale -> Rotate
35         # glTranslatef(300, 200, 0)
36         glTranslatef(300, 200, 0)
37         # glScalef(2.0, 2.0, 1.0)
38         glScalef(2.0, 2.0, 1.0)
39         # glRotatef(30.0, 0, 0, 1)
40         glRotatef(30.0, 0, 0, 1)
41
42         # TODO: draw first ellipse (fill then outline)
43         # self.draw_ellipse_filled(0, 0, 100, 50, color=(0.1,0.6,1.0,0.35))
44         self.draw_ellipse_filled(0, 0, 100, 50, color=(0.1,0.6,1.0,0.35))
45         # self.draw_ellipse_outline(0, 0, 100, 50, color=(0,0,1), width=2)
46         self.draw_ellipse_outline(0, 0, 100, 50, color=(0,0,1), width=2)
47
48         # TODO: second ellipse (different radii/color)
49         # self.draw_ellipse_filled(0, 0, 50, 100, color=(1.0,0.2,0.6,0.35))
50         self.draw_ellipse_filled(0, 0, 50, 100, color=(1.0,0.2,0.6,0.35))
51         # self.draw_ellipse_outline(0, 0, 50, 100, color=(0.6,0.0,0.3), width=2)
52         self.draw_ellipse_outline(0, 0, 50, 100, color=(0.6,0.0,0.3), width=2)
53
54         # TODO: caption
55         # self.draw_text(0, 0, "Hello 2D", color=(0,0,0))
56         self.draw_text(0, 0, "Hello 2D", color=(0,0,0))
57
58         glDisable(GL_BLEND)
59         glPopMatrix()
60
61 if __name__ == "__main__":
62     App(640, 480, b"Ex5 - Hello 2D")
63     glutMainLoop()
64

```

Result:



Part 2: Homework

Exercise 1: Background Practice with PyOpenGL

Part 1: Background color


```
39 def _set_up_opengl(self) -> None:
40     # Mode 1 = solid color
41     if self.mode == 1:
42         # TODO(Part 1): Change these numbers to different RGB values
43         # Example: glClearColor(1, 0, 0, 1) → red background
44         # Red background
45         glClearColor(1, 0, 0, 1)
46         # Green background
47         glClearColor(0, 1, 0, 1)
48         # Blue background
49         glClearColor(0, 0, 1, 1)
50         # Gray background
51         glClearColor(0.5, 0.5, 0.5, 1)
52         pass
53     else:
54         glClearColor(0, 0, 0, 1) # hidden behind image
55         glEnable(GL_TEXTURE_2D)
```

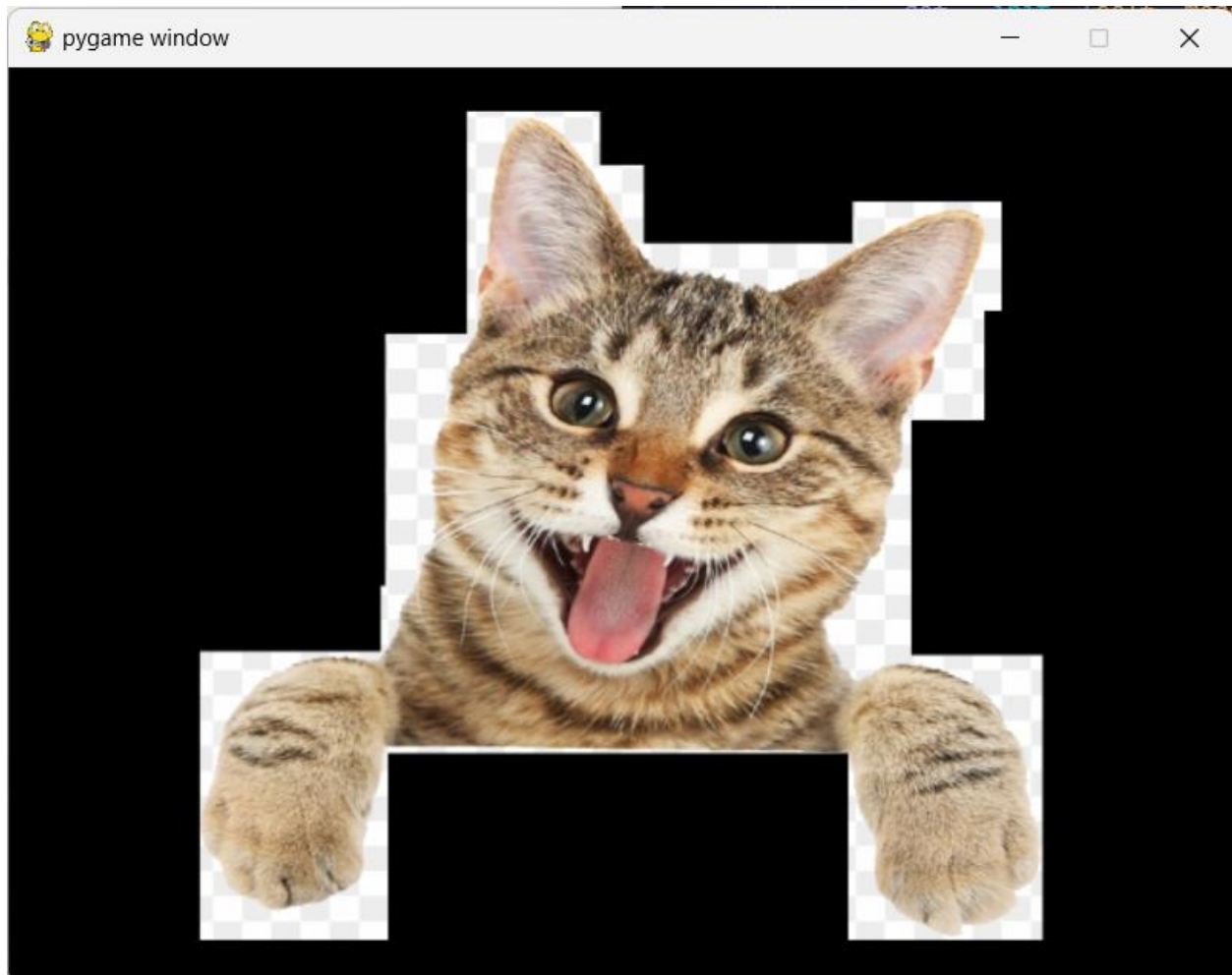


Part 2: Background Image

```
def __init__(self, mode=1):
    self.mode = mode    # 1 = solid color, 2 = image background

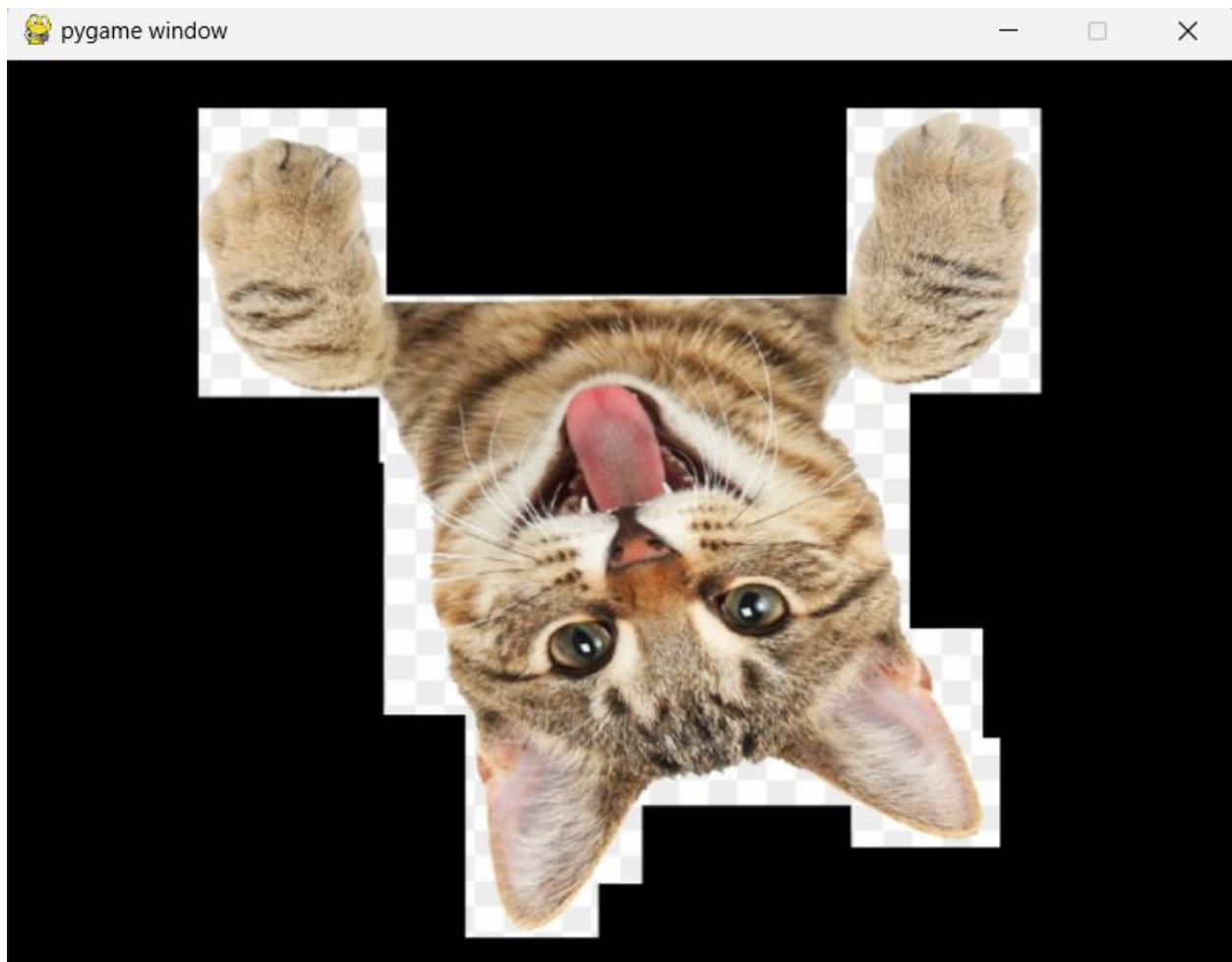
    self._set_up_pygame()
    self._set_up_timer()
    self._set_up_opengl()

    # Load image only if in mode 2
    if self.mode == 2:
        # TODO(Part 2): replace with your image file
        # use self._load_texture("background.jpg") for example
        self.bg_tex = self._load_texture("cat.png")
    pass
```



(optional): Flipping texture coordinates

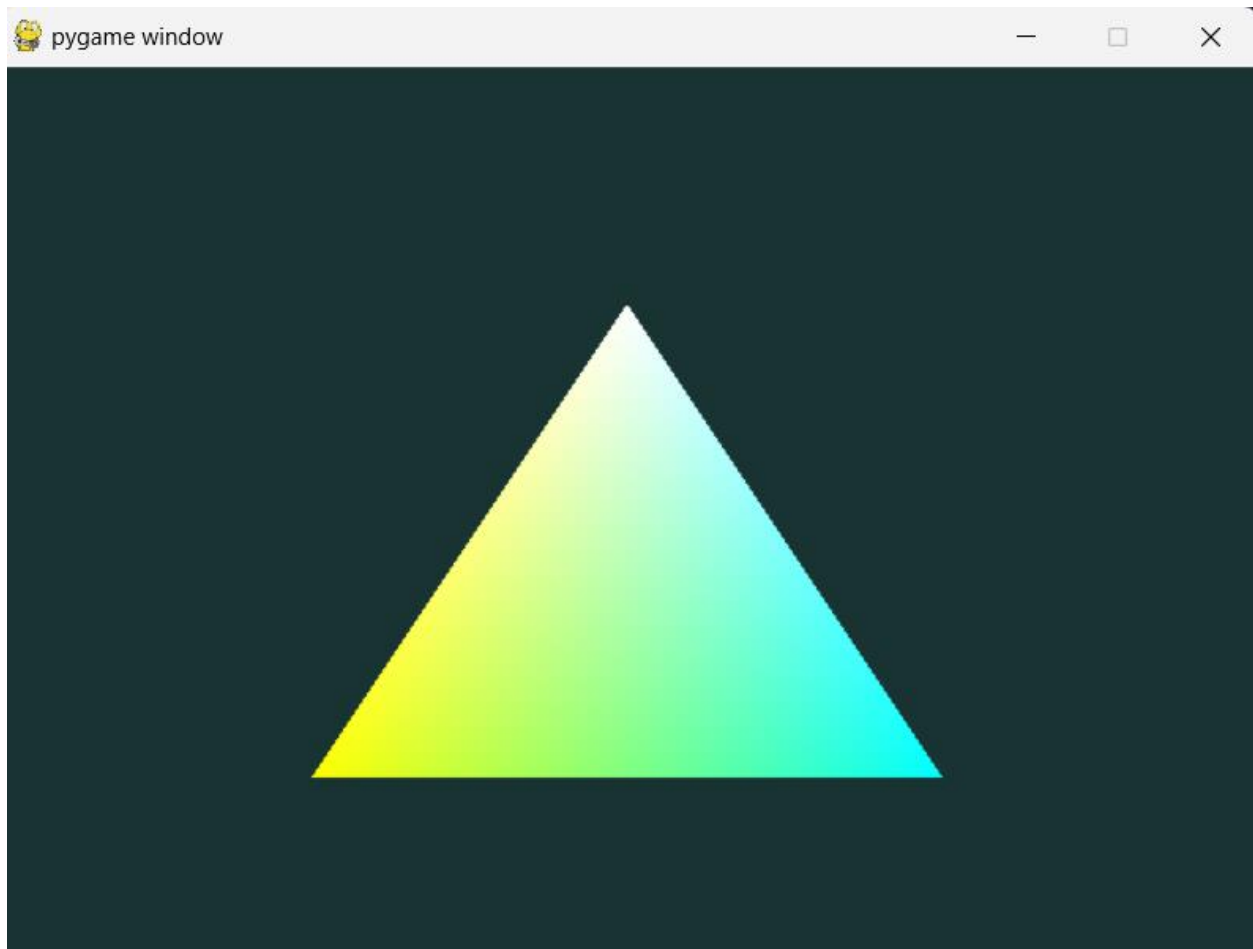
```
71     def _draw_background(self):
72         """Draw textured quad covering the screen"""
73         # TODO(Part 2, Optional Challenge): Try flipping
74         # (swap top and bottom) to see how it affects the
75         glBindTexture(GL_TEXTURE_2D, self.bg_tex)
76         glBegin(GL_QUADS)
77         glTexCoord2f(0, 1); glVertex2f(-1, -1)
78         glTexCoord2f(1, 1); glVertex2f(1, -1)
79         glTexCoord2f(1, 0); glVertex2f(1, 1)
80         glTexCoord2f(0, 0); glVertex2f(-1, 1)
81         glEnd()
```



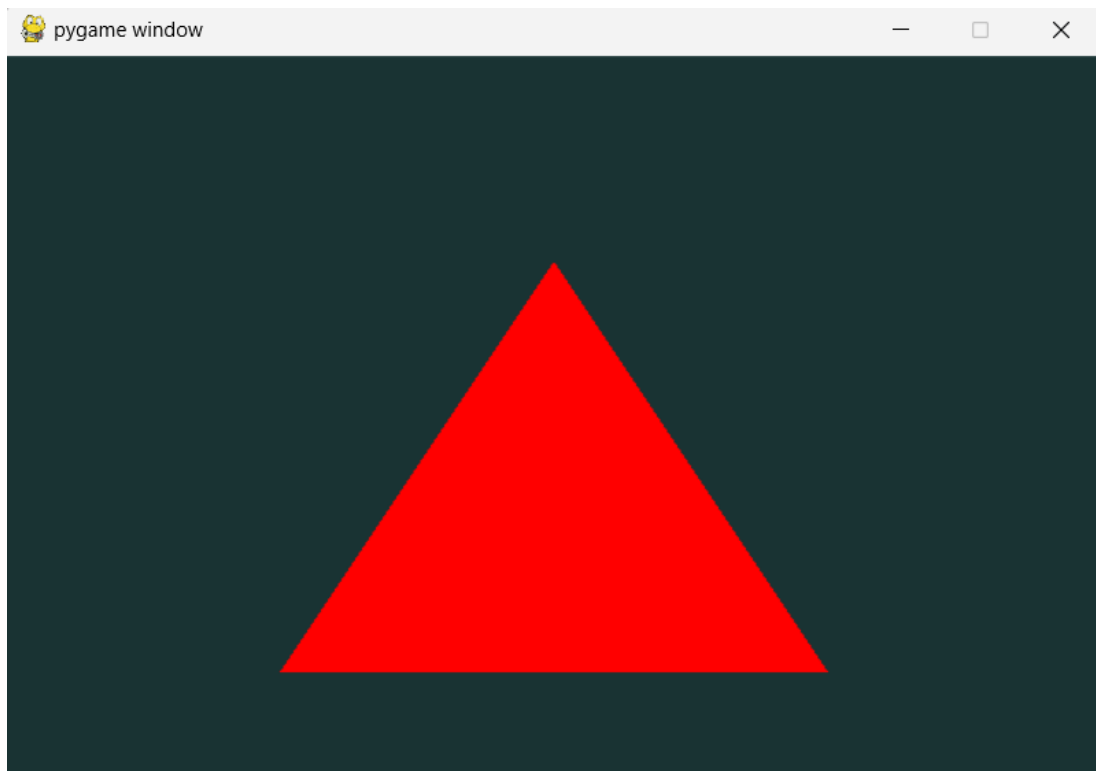
Exercise 2: Introduction to Shaders with PyOpenGL + Pygame

Task 1: Change Vertex Colors

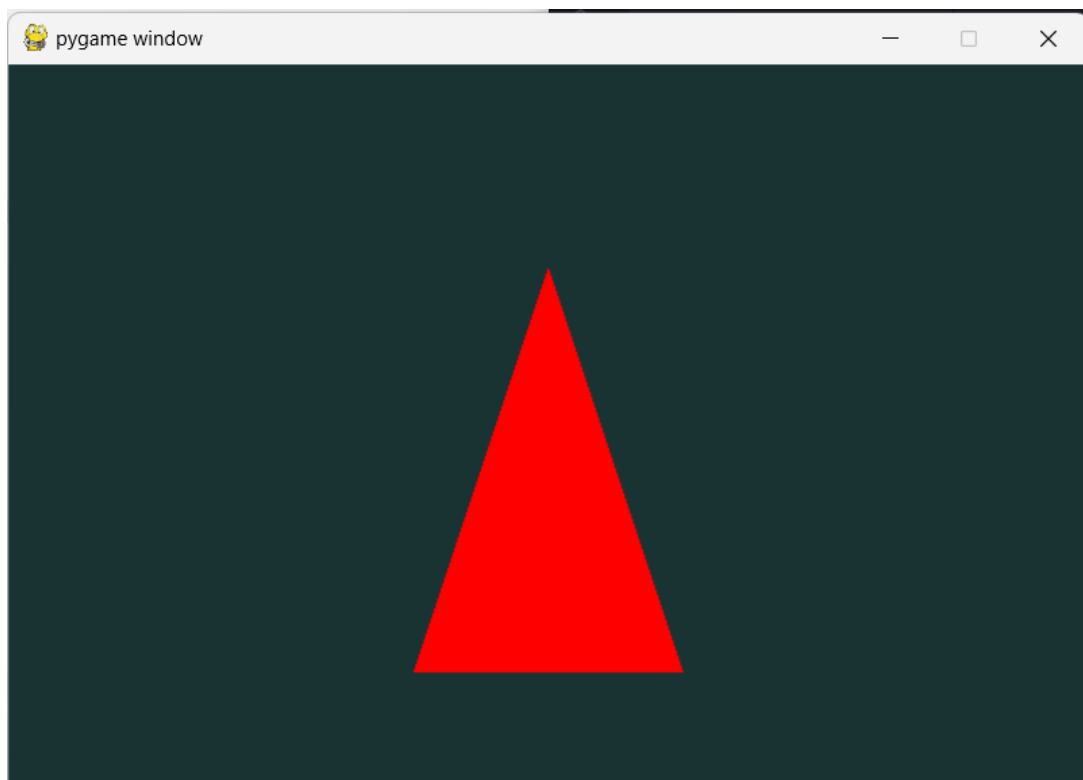
```
121     def __init__(self):
122         """
123         Initialize a triangle.
124         """
125
126         # x, y, z, r, g, b
127         vertices = [
128             -0.5, -0.5, 0.0, 1.0, 1.0, 0.0,
129             0.5, -0.5, 0.0, 0.0, 1.0, 1.0,
130             0.0, 0.5, 0.0, 1.0, 1.0, 1.0
131         ]
```



Task 2: Modify the Fragment Shader

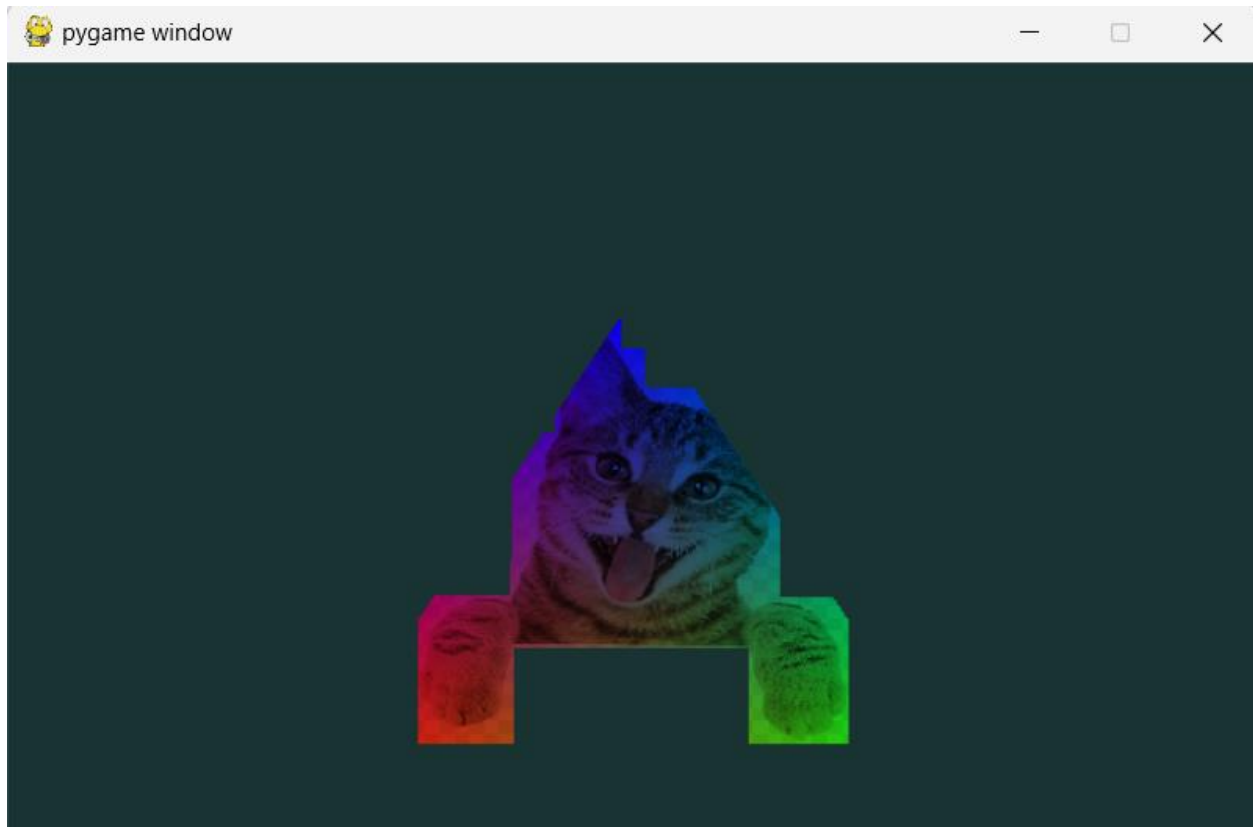


Task 3: Apply Transformations in the Vertex Shader



Exercise 3: Textured Triangle with Shaders (PyOpenGL + Pygame)

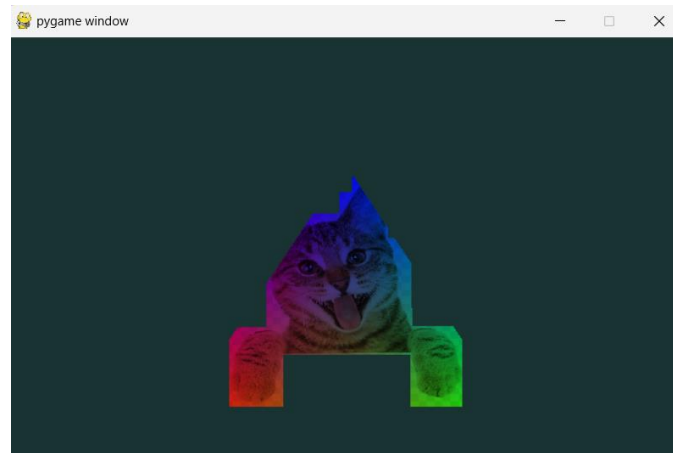
Task 1: Change the Texture



Task 2: Experiment with Texture Coordinates

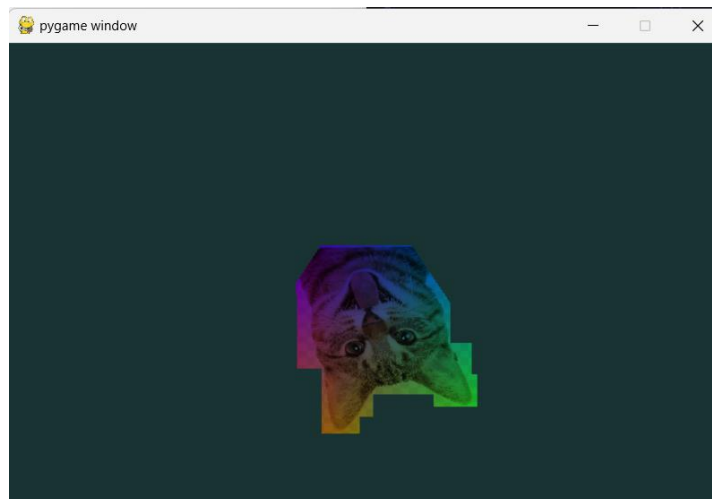
- Horizontal Flip

```
143     vertices = (  
144         -0.5, -0.5, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0,  
145         0.5, -0.5, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0,  
146         0.0, 0.5, 0.0, 0.0, 0.0, 1.0, 0.5, 0.0  
147     )
```



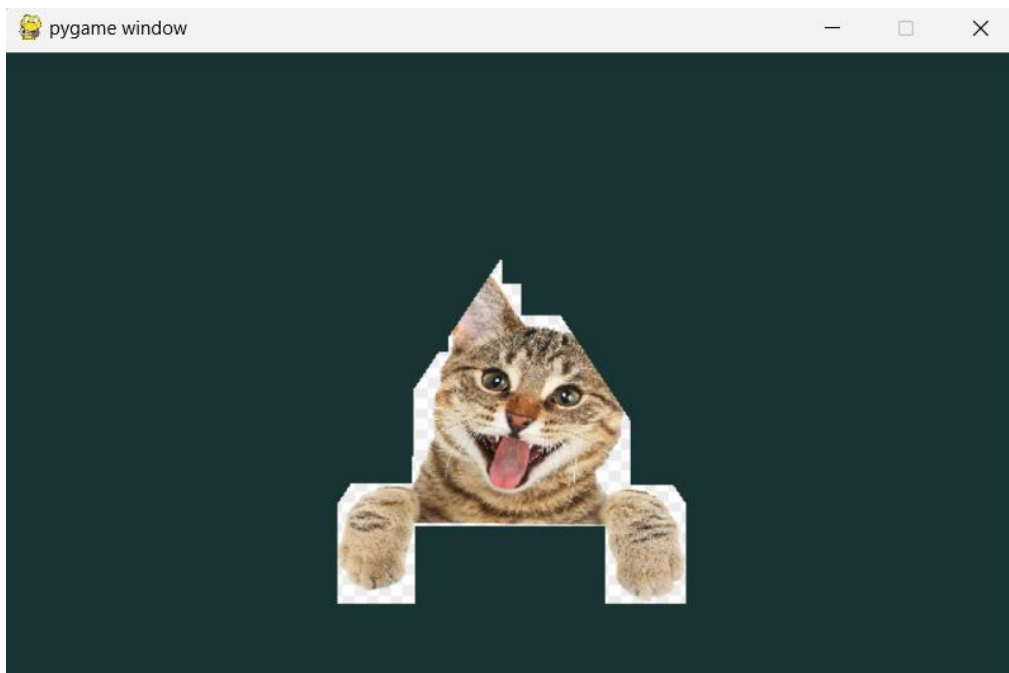
- Vertical Flip

```
143     vertices = (  
144         -0.5, -0.5, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0,  
145         0.5, -0.5, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0,  
146         0.0, 0.5, 0.0, 0.0, 0.0, 1.0, 0.5, 1.0  
147     )
```

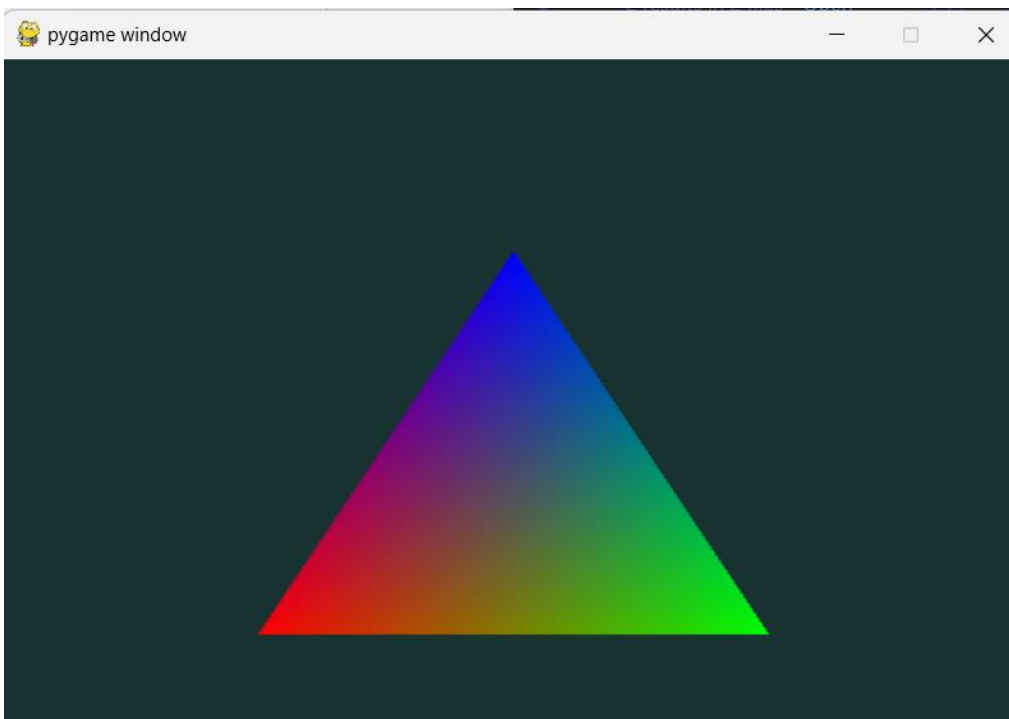


Task 3: Modify the Fragment Shader

- Ignore vertex colors

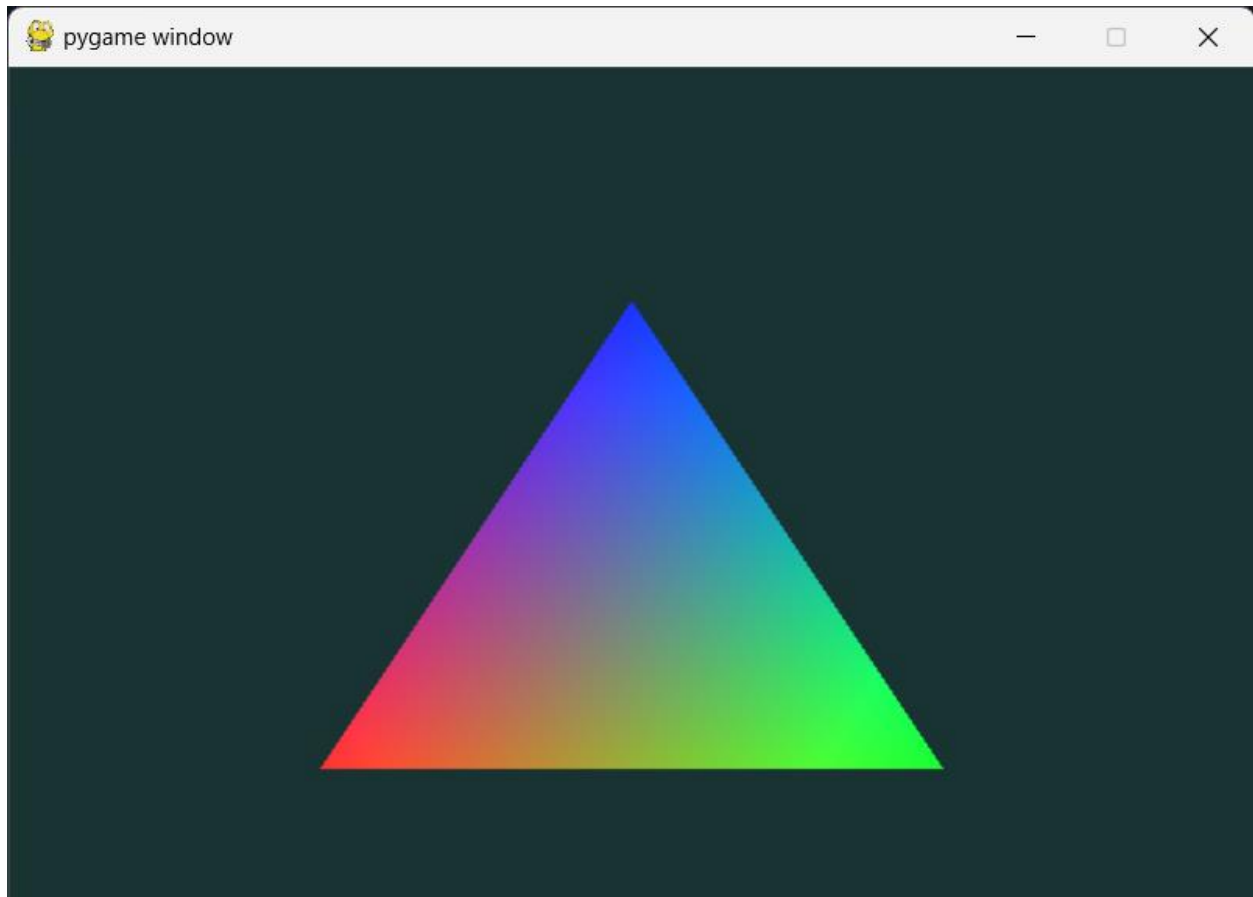


- Ignore the texture

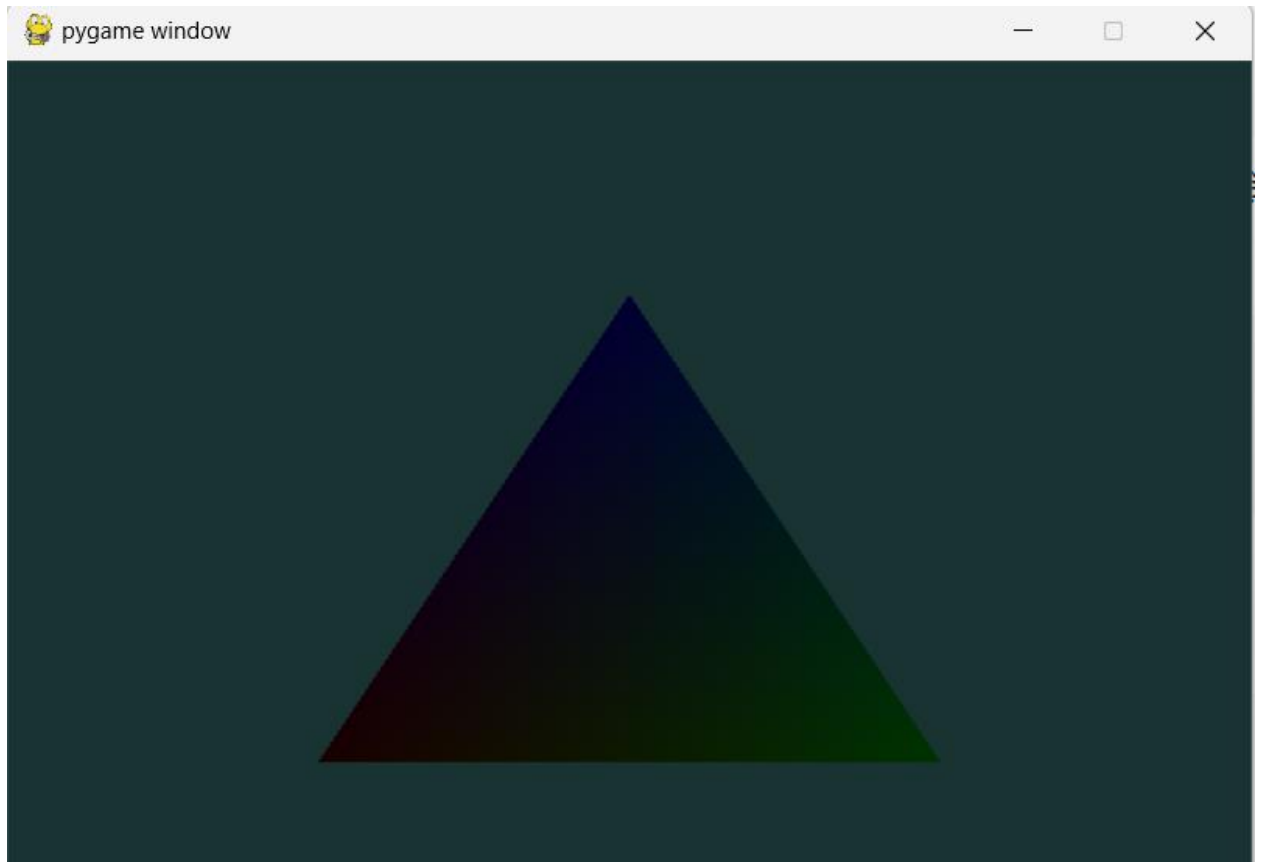


Task 4: Bonus Challenge

- `glBlendFunc(GL_SRC_ALPHA, GL_ONE)`: Make the color brighter

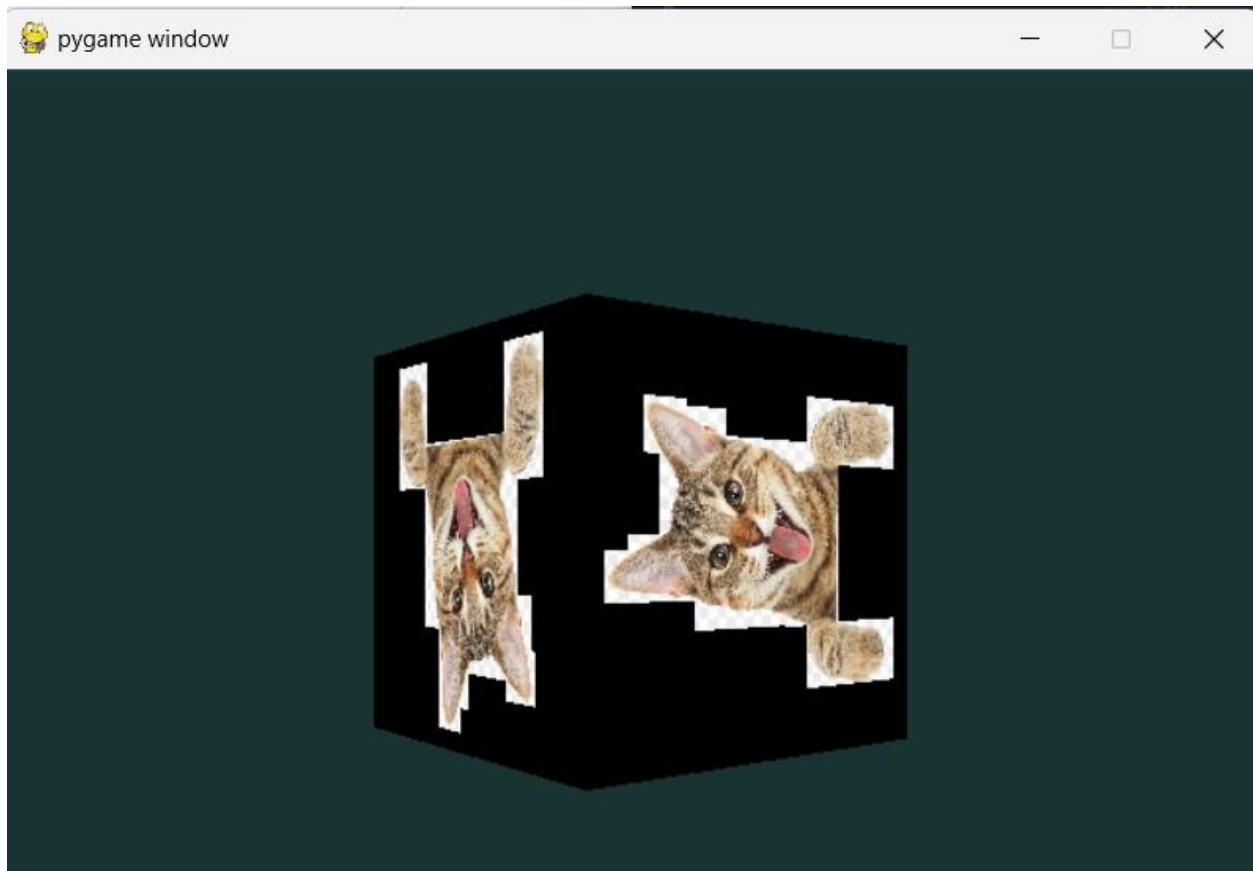


- `glBlendFunc(GL_DST_COLOR, GL_ZERO)`: Make more layer for darker



Exercise 4: Spinning Textured Cube (PyOpenGL + Pygame)

Task 1: Change the Texture



Task 2: Modify Rotation Speed

```
61 self.eulers[1] += 0.75
```

➔ The cube rotates faster.

Otherwise,

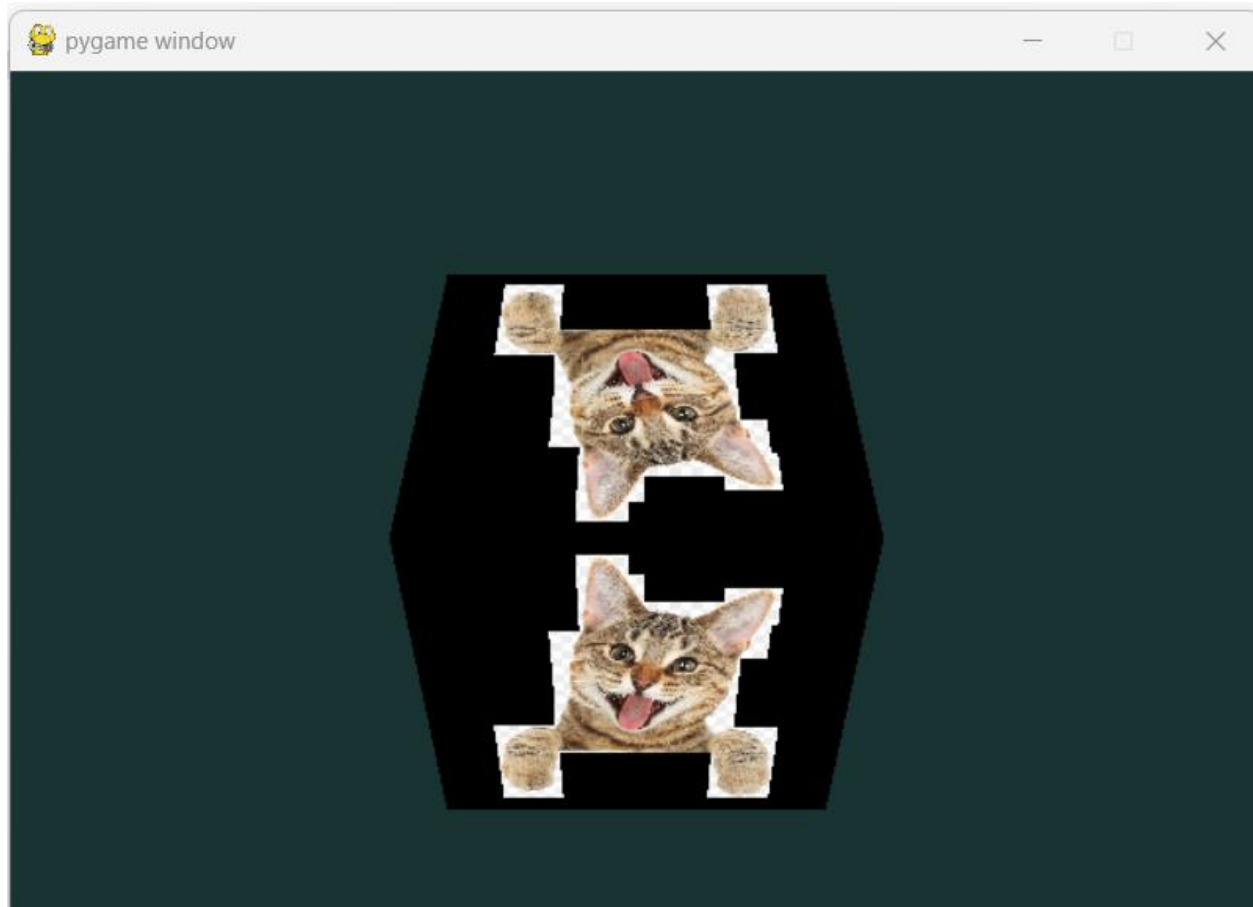
```
61 self.eulers[1] += 0.1
```

➔ The cube rotates slower.

Task 3: Apply Different Transformations

Change the rotates direction:

```
m2=pyrr.matrix44.create_from_axis_rot  
    axis = [1, 0, 0],  
    theta = np.radians(self.eulers[1]  
    dtype = np.float32  
)
```



```

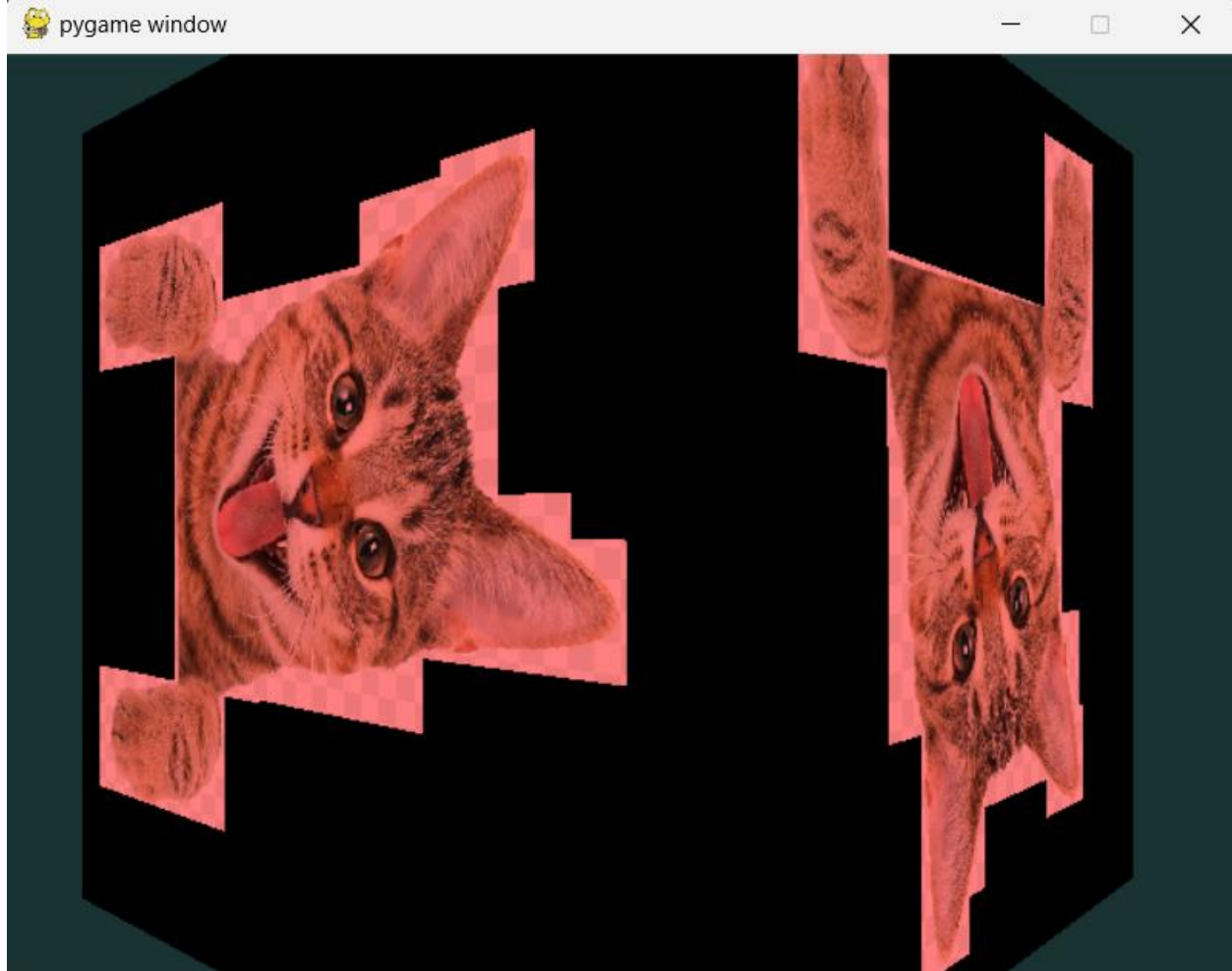
71
72     model_transform = pyrr.matrix44.create_identity(dtype=np.float32)
73
74     scale_factor = 2.0
75     scale_matrix = pyrr.matrix44.create_from_scale(
76         scale=np.array([scale_factor, scale_factor, scale_factor], dtype=np.float32)
77     )
78
79     model_transform = pyrr.matrix44.multiply(
80         m1=model_transform,
81         m2=scale_matrix
82     )
83
84     model_transform = pyrr.matrix44.multiply(
85         m1=model_transform,
86         m2=pyrr.matrix44.create_from_axis_rotation(
87             axis = [0, 1, 0],
88             theta = np.radians(self.eulers[1]),
89             dtype = np.float32
90         )
91     )

```

pygame window



Task 4: Experiment with Shaders



Task 5: Bonus Challenge

- Pymarid: Change the vertices



```
1
2 # x, y, z, s, t
3 vertices = (
4     -0.5, 0.0, -0.5, 0, 0,
5     0.5, 0.0, -0.5, 1, 0,
6     0.5, 0.0, 0.5, 1, 1,
7
8     0.5, 0.0, 0.5, 1, 1,
9     -0.5, 0.0, 0.5, 0, 1,
10    -0.5, 0.0, -0.5, 0, 0,
11
12    -0.5, 0.0, -0.5, 0, 0,
13    0.5, 0.0, -0.5, 1, 0,
14    0.0, 0.8, 0.0, 0.5, 1,
15
16    0.5, 0.0, -0.5, 0, 0,
17    0.5, 0.0, 0.5, 1, 0,
18    0.0, 0.8, 0.0, 0.5, 1,
19
20    0.5, 0.0, 0.5, 0, 0,
21    -0.5, 0.0, 0.5, 1, 0,
22    0.0, 0.8, 0.0, 0.5, 1,
23
24    -0.5, 0.0, 0.5, 0, 0,
25    -0.5, 0.0, -0.5, 1, 0,
26    0.0, 0.8, 0.0, 0.5, 1
27 )
```



- Rectangle: Adjust the axis



```
1 vertices = np.array(vertices, dtype=np.float32)
2 vertices[:,5] *= 1.5
3 vertices[2::5] *= 0.5
```